Replicating and Extending Prefix-Tuning: Table-to-Text Generation on E2E, ViGGO and a New Product-Description Dataset

Niharika Kunaminna
Student ID 48930520
COMP8240 Applications of Data Science, Macquarie University
niharika.kunaminna@students.mq.edu.au

Abstract

I replicate the table-to-text setting of prefix-tuning (Li and Liang, 2021), which freezes a pretrained language model and trains only a short continuous prefix, and then test how far the method carries to new data. After five fixes to its 2021 environment, the authors' own code with GPT-2 medium reaches 67.4 BLEU on the E2E test set after one training epoch on a free Colab GPU (the paper uses five) and with the paper's general rather than its E2E-specific hyperparameters, 2.3 BLEU below the 69.7 reported. A lightweight reimplementation with GPT-2 small and Hugging Face `peft` on a laptop CPU reaches 57.1 BLEU and is used for the new-data experiments, so those results test this reimplementation rather than the authors' code. On ViGGO, an existing video-game dataset, a prefix warm-started from E2E beats one trained from scratch (38.3 vs. 32.7 BLEU, 91% vs. 76% slot coverage). On a new Flipkart product-description dataset that I built and annotated (131 pairs), the method largely fails (3.8 BLEU), and annotation shows why: most human descriptions contain facts that are not in the input. Two annotators agreed only slightly ($\kappa = 0.03$ over 100 pairs, no written guidelines). With written guidelines, agreement on 31 further pairs was substantial ($\kappa = 0.65$), but 30 and 29 of the 31 labels were *no*, so this is not evidence that the guidelines work. An LLM-as-judge pass shows that word-overlap metrics and slot coverage miss many hallucinations.

1 Description of the Paper and Justification

1.1 The paper

Fine-tuning adapts a pretrained language model (LM) by updating all of its parameters, so a full copy of the model must be stored for each task. Li and Liang (2021) propose *prefix-tuning*: the LM is frozen and a short sequence of continuous, task-specific vectors (the *prefix*) is prepended to the activations at every transformer layer. Real tokens attend to the prefix as if it were a sequence of "virtual tokens". Only the prefix is trained, about 0.1% of the parameters of the model. To stabilise training, the prefix is not optimised directly but produced by a small MLP from a smaller matrix; after training only the prefix is kept. The paper applies the method to table-to-text generation with GPT-2 (Radford et al., 2019) and to summarisation with BART (Lewis et al., 2020).[1]

1.2 Evaluation framework and original results

Table-to-text output is scored against human references with the official scripts of each benchmark: BLEU (Papineni et al., 2002), NIST (Doddington, 2002), METEOR (Banerjee and Lavie, 2005), ROUGE-L (Lin, 2004) and CIDEr (Vedantam et al., 2015) for E2E, plus TER and learned metrics for WebNLG and DART; summarisation is scored with ROUGE-1/2/L. The paper compares prefix-tuning with full fine-tuning, fine-tuning only the top two layers, and adapter-tuning (Houlsby et al., 2019).

With GPT-2 medium and 0.1% of the parameters, prefix-tuning matched or beat fine-tuning on table-to-text: 69.7 vs. 68.2 BLEU on E2E, 46.4 vs. 46.2 on DART, and 55.1 vs. 46.5 BLEU on WebNLG overall, where the gain came mostly from categories unseen in training (45.6 vs. 27.7). On XSum it was slightly worse than fine-tuning (ROUGE-L 35.05 at 0.1% and 36.05 at 2% vs. 37.25). It also beat fine-tuning in low-data settings by 2.9 BLEU on average, and extrapolated better to unseen topics. Table 1 lists the E2E numbers I compare against.

1.3 Justification

Venue. ACL is the flagship venue of computational linguistics, ranked A* in the CORE confer-

[1] Paper: https://aclanthology.org/2021.acl-long.353/; official code: https://github.com/XiangLi1999/PrefixTuning.

ence rankings and the top Computational Linguistics venue in Google Scholar Metrics.[2]

Impact. The paper has over 8,000 citations on Google Scholar (checked 2 November 2026). It is one of the founding methods of parameter-efficient fine-tuning; later methods such as prompt tuning (Lester et al., 2021) and LoRA (Hu et al., 2022) compare against it.

Reproducibility. The authors released code, all four datasets are public with standard splits, and the method is reimplemented in the widely used Hugging Face `peft` library (Mangrulkar et al., 2022), which lowers the risk of the project.

2 Datasets in the Original Paper

E2E (Novikova et al., 2017b) is a crowdsourced restaurant-domain dataset of about 50K examples. Each input is a meaning representation (MR): a flat list of up to eight attribute–value pairs (*name, eatType, food, priceRange, customer rating, area, familyFriendly, near*). Each MR has several human references; in the official split, 42,061 training pairs, 4,672 development pairs and 4,693 test pairs, where the test set has 630 distinct MRs (7.4 references each). Figure 1 shows a test example.[3]

WebNLG (Gardent et al., 2017) has about 22K examples whose inputs are sets of RDF triples (*subject | property | object*) taken from DBpedia, verbalised as one or more sentences. The test set is split into categories seen in training and five unseen categories, which is what allows the paper's extrapolation results.[4]

DART (Nan et al., 2021) is an open-domain dataset of about 82K examples, also expressed as triples, built from Wikipedia tables (WikiSQL, WikiTableQuestions) together with converted E2E and WebNLG data.[5]

XSum (Narayan et al., 2018) pairs about 225K BBC news articles (431 words on average) with one-sentence summaries (23.3 words), and is used for the BART summarisation experiments.[6]

[2] https://portal.core.edu.au/conf-ranks/; https://scholar.google.com/citations?view_op=top_venues&vq=eng_computationallinguistics.

[3] https://github.com/tuetschek/e2e-dataset

[4] https://gitlab.com/shimorina/webnlg-dataset

[5] https://github.com/Yale-LILY/dart

[6] https://github.com/EdinburghNLP/XSum

3 Replication of the Original Work

Scope. I replicated the paper's E2E experiments only. E2E is the table-to-text benchmark the paper reports in most detail (all five metrics and the low-data experiments), and with roughly one training epoch per free Colab session, adding WebNLG, DART or the BART summarisation runs on XSum was not feasible. The replication has two parts: the authors' own code with GPT-2 medium (Section 3.1), which tests whether the reported result can be reproduced, and a lightweight `peft` reimplementation with GPT-2 small (Section 3.2), which runs on a laptop CPU and is the system used for the new-data experiments in Section 5.

3.1 Running the authors' code

The specification asks for the original code on the original data. The official repository (branch `cleaned`) trains E2E prefixes with `gpt2/train_e2e.py`, which builds a command for `run_language_modeling.py`, on top of a modified copy of `transformers` 3.2.0 bundled in the repository; `gen.py` and `run_generation.py` then decode and call the official E2E scorer. I ran it on Google Colab with a T4 GPU (notebook `colab/original_code_run_with_outputs.ipynb`, with all cell outputs). Getting it to run took five fixes:

1. Colab's Python 3.13 cannot build `tokenizers` 0.8.1rc2, which the bundled `transformers` requires. I created a Python 3.8.20 environment with `uv` and installed torch 1.13.1, the bundled library and `six`, which the library imports but does not declare.
2. `train_e2e.py` launches training with `os.system("python ...")`, which picked up Colab's own Python and current `transformers` (`ImportError: cannot import name 'cached_path'`). I called the Python 3.8 interpreter directly.
3. Model cache, test data, output folder and scorer are hard-coded to the authors' servers (`/u/scr/xlisali/...`). I downloaded GPT-2 medium from Hugging Face and patched the paths.
4. `gen.py` infers its settings from the name of the model folder: a folder without `prefixtune` in its name crashes with

E2E (test)
`name[The Vaults], eatType[restaurant], food[French], priceRange[moderate], area[riverside], familyFriendly[yes], near[Raja Indian Cuisine]`
Ref: The Vaults is a reasonably priced, family friendly restaurant located near Raja Indian Cuisine by the river. (1 of 11 refs)
Ours: The Vaults is a French restaurant near Raja Indian Cuisine. It is family friendly and has a moderate price range.

WebNLG (as included in the DART test set)
`(Ab Klink | ALMA_MATER | Erasmus University Rotterdam), (Ab Klink | PARTY | Christian Democratic Appeal), (Ab Klink | BIRTH_PLACE | Netherlands)`
Ref: Ab Klink was born in the Netherlands, belongs to the Christian Democratic Appeal party and graduated from Erasmus University in Rotterdam.

DART (test, from WikiTableQuestions)
`(Hawaii Five-O | NOTES | Episode: The Flight of the Jewels), ([TABLECONTEXT] | [TITLE] | Jeff Daniels), ([TABLECONTEXT] | TITLE | Hawaii Five-O)`
Ref: Jeff Daniels played in the Hawaii Five-O episode The Flight of the Jewels

ViGGO (test)
`inform(name[The Witcher 3: Wild Hunt], developer[CD Projekt RED], esrb[M (for Mature)], rating[excellent], genres[action-adventure, role-playing])`
Ref: The Witcher 3: Wild Hunt is CD Projekt RED's excellent action-adventure title. Rated M (for Mature), it has been universally praised for its role-playing elements. The game is available for PlayStation, Xbox, and PC.

Flipkart (new, train)
`name[Nimya Solid Men's Polo Neck T-Shirt], category[Clothing], subcategory[Men's Clothing], priceRange[expensive], type[Polo Neck], ideal for[Men's], occasion[Casual], pattern[Solid]`
Ref: Nimya Is In Lifestyle Fashion And Athletic Apparels. This Polo T-Shirt Is Made To Bring Out The Style In You. (annotated *yes*)

Figure 1: Examples from the datasets: E2E (with my E2E model's output), WebNLG and DART triples from the paper, and the two datasets I added. XSum is not shown: each instance is a full BBC article (431 words on average) with a one-sentence summary. The ViGGO reference mentions platforms that are not in the MR.

`NameError: name 'tuning_mode' is not defined`, and it always passes a prefix length of 20. I called `run_generation.py` directly with the training settings.

5. `run_generation.py` in turn reads the prefix format (`cat`, `peek`, ...) from the folder name and otherwise stops with an `AssertionError`. I linked the model folder under a name containing `cat`.

Training then ran with GPT-2 medium and the MLP reparameterisation (hidden size 512; 25,750,016 parameters during training). This run deviates from the paper's E2E configuration in two ways. It uses the paper's general table-to-text settings (its Section 5.3: learning rate 5×10^{-5} with linear decay, batch size 5, prefix length 10) rather than the E2E-specific settings in its appendix Table 5 (learning rate 8×10^{-5}, batch size 10, prefix length 5), which I identified only after the run; and it trains for one epoch rather than five, because of Colab's session limits. Section 3.3 treats both as untested causes of the remaining gap. A first 300-step test (1,500 examples, 3.6% of an epoch) took about a minute; the full epoch over all 42,061 training pairs (8,413 steps) took 31 minutes on the T4, against about an hour for my much smaller CPU run, with the logged loss falling from 73.8 at step 500 to 28.8 at step 8,000 (the code logs a summed, not per-token, loss). After training only the prefix activations are needed: $10 \times 24 \times 2 \times 1024 = 491{,}520$ values, 0.14% of GPT-2 medium.

Result. Decoding the 630 test MRs with beam search (beam 5) and scoring with the same scorer as my `peft` runs gave **67.4 BLEU**, 7.24 NIST and 68.8 ROUGE-L (Table 1), 2.3 BLEU below the paper's 69.7, after one epoch where the paper's E2E run used five. Slot coverage was 98.3%: the area appears in 97% of outputs (544/558), food in 96% and price in 95%, against 23%, 73% and 62% for my `peft` model. The authors' code therefore reproduces the paper closely, and the remaining gap is consistent with the shorter training and the different hyperparameters; Section 3.3 shows that the scorer explains the NIST gap but not the BLEU gap.

3.2 Reimplementation with `peft`

Environment. All runs used a Windows laptop CPU (no GPU), Python 3.11 with torch 2.14.1, transformers 5.18.0, peft 0.21.2 and datasets 5.0.1. The university VM and Codespace planned in the

proposal were not needed once the CPU run time was known (about one hour for the main E2E run). One practical issue: installing into the Microsoft Store Python failed with "filename too long", fixed by a virtual environment on a short path. Code and results are public.[7]

Setup. `train_prefix.py` wraps GPT-2 small (124M parameters) with `peft`'s `PrefixTuningConfig` and a 10-token prefix: 184,320 trainable of 124,624,128 parameters (0.148%), close to the paper's 0.1%. Each MR is linearised (`name : The Vaults | food : French | ...==>`) and the loss is computed on the reference tokens only. I trained on a random 4,000 of the 42,061 E2E training pairs for 2 epochs (batch 8, AdamW, learning rate 3×10^{-2} with linear decay) and decoded greedily. A pilot on 1,000 pairs chose the learning rate: 10^{-2} gave 13.4 BLEU on 50 test MRs, 3×10^{-2} gave 47.5. A larger run (8,000 pairs, 3 epochs) was stopped after 380 of 3,000 steps because it would have taken over two hours on the CPU.

Scoring. I score all 630 distinct test MRs against all their references with my own scorer: NLTK BLEU, NIST and METEOR on lower-cased tokens, and `rouge_score` ROUGE-L (best over references). CIDEr is not computed. A first evaluation on the first 200 test MRs gave 55.7 BLEU, but those MRs cover only 10 of the 18 restaurant names, so I re-scored the saved prefix on the full set. Re-running from the saved prefix reproduces the numbers exactly. I also report *slot coverage*: the share of *name, eatType, food, priceRange, area* and *near* values that appear word for word in the output, computed the same way for the human references as a baseline.

3.3 How close, and why not closer

With the authors' code the replication is close: 67.4 BLEU against 69.7, 7.24 NIST against 8.81, and 68.8 ROUGE-L against 71.4 (Table 1). Three differences remain: one training epoch instead of five, the general rather than the E2E-specific hyperparameters, and my scorer instead of the official E2E scripts (Dušek et al., 2020). The authors'-code outputs were lost with the Colab session, so I measured the scorer difference on my own greedy `peft` outputs instead: the official script gives 56.3 BLEU and 6.92 NIST where my scorer gives 57.1 and 5.17 (ROUGE-L 59.9 vs. 59.4; `results/e2e_full/official_e2e_metrics.json`). Official NIST is 1.7 higher than mine, about the size of the NIST gap to the paper (7.24 vs. 8.81), so the NIST gap is most likely a scorer artefact. Official BLEU is 0.8 *lower*, so the scorer does not explain the BLEU gap: if the same offset holds for the authors'-code output, it would score about 66.6 officially, two to three points below the paper. I attribute this gap to the shorter training and different hyperparameters, which I did not test. All results in this report are single runs with one random seed, so I cannot report run-to-run variance. I therefore interpret only the large differences (2.3 BLEU to the paper, 10.3 between the two systems, 5.6 from warm-starting on ViGGO) and treat differences below about one BLEU point as within noise.

My `peft` reimplementation is 12.6 BLEU, 3.6 NIST and 12.0 ROUGE-L below the paper. The method clearly works, since 0.15% of the parameters turn a general LM into a fluent restaurant

System	BLEU	NIST	MET	R-L	CIDEr
Reported by Li and Liang (2021)					
GPT-2 M, fine-tune	68.2	8.62	46.2	71.0	2.47
GPT-2 M, prefix 0.1%	69.7	8.81	46.1	71.4	2.49
GPT-2 L, prefix	70.3	8.85	46.2	71.7	2.47
This work (own scorer, see text)					
GPT-2 M, orig. code, 1 ep.	67.4	7.24	78.8†	68.8	–
GPT-2 S, `peft`, 0.15%	57.1	5.17	64.9†	59.4	–
This work, official E2E scorer					
GPT-2 S, `peft`, 0.15%	56.3	6.92	35.5	59.9	1.63

Table 1: E2E test set (630 MRs). The first two rows of this work use my NLTK/`rouge_score` scorer; the last row scores the same `peft` outputs with the official E2E scripts. The authors'-code outputs were not kept, so they could not be re-scored. †NLTK METEOR is not comparable with the official METEOR (it is higher here although every other metric is lower).

Setting	Paper	This work
Model	GPT-2 medium/large	GPT-2 small
Training pairs	42,061 (all)	4,000 (9.5%)
Epochs / batch	5 / 10	2 / 8
Learning rate	8×10^{-5}	3×10^{-2}
Prefix length	5	10
MLP reparam.	yes	no (`peft` default)
Decoding	beam 5	greedy
Scorer	official E2E	NLTK, rouge-score
Hardware	GPU	laptop CPU

Table 2: Differences between the paper's E2E setup (its appendix Table 5) and my `peft` run.

[7] https://github.com/KunaminnaNiharika/prefix-tuning-replication

Slot	n	Model	Human refs
name, near	630, 618	100%	100%
eatType	630	100%	98%
food	546	73%	88%
priceRange	480	62%	62%
area	558	**23%**	78%

Table 3: E2E slot coverage (value appears verbatim) on the 630 test MRs. With a looser check ("riverside" or "centre/center" anywhere) area rises to 86% for the references but stays at 23% for the model.

describer, but it is not a like-for-like replication. Table 2 lists the differences. Candidate causes, of which decoding (below) and the scorer were tested: (1) GPT-2 small has a third of the parameters of GPT-2 medium; (2) I used under a tenth of the training data for two of the paper's five epochs; (3) without the paper's MLP reparameterisation, training the prefix directly needed a learning rate more than two orders of magnitude higher, and the paper reports that direct optimisation is less stable and performs worse; (4) greedy decoding instead of beam search; (5) the scorer differs: tokenisation, NIST and METEOR implementations differ from the official E2E scripts (Dušek et al., 2020), which is visible in METEOR being *higher* than the paper's; on these outputs the official script gives 0.8 BLEU less but 1.7 NIST more than mine (above).

Decoding ablation. To test one of these causes directly, I re-decoded the saved E2E prefix in a fresh Colab environment (T4 GPU, transformers 5.22.0, peft 0.21.2). Greedy decoding reproduced my laptop numbers exactly (57.1 BLEU), and beam search with 5 beams on the same prefix raised BLEU to 59.4 (+2.3), NIST to 6.06, ROUGE-L to 62.3 and slot coverage from 78.1% to 83.2%, with the area now mentioned in 45% of outputs instead of 23%. Decoding therefore explains about 2 of the 12.6 BLEU points; the remaining causes were not tested separately.

Error analysis. Slot coverage (Table 3) shows where the gap comes from: the model always copies the name, type and landmark but mentions the area in only 23% of outputs, against 78% for the human references, and often drops the food type. It also produces a fixed nonsense phrase, "has a customer rating of no children", in 21 of the 630 outputs (3.3%), which merges *familyFriendly[no]* with the rating template. Omissions make outputs shorter than the references, which both BLEU's brevity penalty and ROUGE-L's recall term penalise.

4 Construction of New Data

4.1 Existing dataset: ViGGO

ViGGO (Juraska et al., 2019) is a video-game dataset for conversational data-to-text generation: 6,900 MR–utterance pairs (5,103 / 714 / 1,083) over 14 game attributes, each MR wrapped in one of 9 dialogue acts (e.g. *inform, request, verify_attribute*). Its slot–value format is the same as E2E's, which makes it a clean test of transfer to a new domain; unlike E2E, about half of its utterances are questions. I downloaded it from the GEM release on Hugging Face (`GEM/viggo`; Gehrmann et al., 2021) with `get_viggo.py`, linearised MRs as for E2E and dropped the dialogue-act wrapper except in one run. For CPU time I used 2,000 training pairs and the first 100 distinct test MRs.[8]

4.2 New dataset: Flipkart product descriptions

Source. The proposal planned a Kaggle e-commerce file, but its product-description field turned out to contain no text (`build_dataset.py` is kept in the repository as a record). I therefore used a 1,000-product sample of the PromptCloud Flipkart product data, redistributed on Hugging Face[9] under CC BY-SA 4.0. Each product has a name, a category tree, a retail price in rupees, a free-text description and a list of specification key–value pairs.

Processing. `build_flipkart.py` turns each product into an E2E-style MR and a target sentence. The MR holds the name, brand, top two category levels, a price band (terciles: up to Rs. 799 *cheap*, up to Rs. 1,500 *moderate*, above that *expensive*) and up to four specification slots, preferring the most frequent keys (*type, ideal for, color, occasion, pattern*) so that slots are shared across products as in E2E. The target is the first two sentences of the description after removing the name line, price line, boilerplate and repeated lines. Table 4 shows how 1,000 products became 131

[8] https://huggingface.co/datasets/GEM/viggo

[9] https://huggingface.co/datasets/kenthua/flipkart-raw-demo, from https://www.kaggle.com/datasets/PromptCloudHQ/flipkart-products

Step	Products
Raw sample	1,000
description only sales boilerplate	−596
description is a spec dump	−190
cleaned description > 45 words	−22
cleaned description < 8 words	−21
duplicate target sentence	−37
no price	−3
Usable pairs (train / dev / test 105 / 13 / 13)	131
After removing 9 pairs annotated *no*	122

Table 4: Construction of the Flipkart dataset (`build_log.json`).

pairs: most descriptions were pure sales text ("Buy ... at Flipkart.com. Only genuine products") or a dump of the specification table. The 131 pairs were split 105 / 13 / 13. Targets average 26.1 words and MRs 8.65 slots (E2E outputs average 22.9 tokens; Li and Liang, 2021). Only 57.3% of targets contain any MR value verbatim, which is the first sign that the data is weakly grounded. Figure 1 shows an example.

4.3 Human annotation and agreement

Because Flipkart descriptions were written to sell, not to verbalise an MR, I annotated 100 randomly chosen pairs for whether the target is supported by its MR: *yes* (no factual claim outside the MR), *partial* (about the right product but adds facts, e.g. a colour or material not in the MR) or *no* (nothing product-specific is supported), with a short note per row. The result was 25 yes, 66 partial and 9 no. The 9 *no* pairs were removed (`filter_by_annotation.py`), leaving 98 / 12 / 12 pairs for the experiments. This filter relies on my labels alone; given the low agreement reported below, it is not a reliable filter, and 31 of the 131 pairs had not been annotated when the filter was applied (they were labelled later, see below). Of the 12 test pairs, 10 were in this annotation (9 *partial*, 1 *yes*) and 2 were not.

A second annotator labelled all 100 pairs independently in two batches of 50, with only the label names as instruction. Agreement was *slight* (Landis and Koch, 1977) in both batches (Cohen's $\kappa = 0.03$ and 0.06; Cohen, 1960) and over all 100 pairs ($\kappa = 0.03$, 30% raw agreement; Table 5). The disagreement was not random, and its direction changed between batches: in the first, the second annotator labelled 13 of my 29 *partial* pairs *no* and 9 *yes*; in the second, 22 of my 37 *partial* pairs *yes* and 8 *no*. For example, "...this black top will make you look elegant" for an MR with no colour: I counted "black" as an unsupported fact, the second annotator judged the description as about the right product. With only the label names given, the boundary of *partial* was undefined, so the annotation measured how differently two careful readers interpret an undefined label, not annotator carelessness.

Pairs	n	Raw agr.	κ	95% CI
First batch	50	32%	0.03	−0.13 to 0.19
Second batch	50	28%	0.06	−0.07 to 0.18
All	100	30%	0.03	−0.07 to 0.13
With guidelines*	31	97%	0.65	0.00 to 1.00

Table 5: Agreement between the two annotators (yes / partial / no). First three rows: the first 100 Flipkart pairs, labelled with only the label names as instruction. *Last row: the 31 remaining pairs, labelled with the written guidelines (`guideline_round_annotator1/2.csv`); 30 and 29 of the 31 labels were *no*, see text. CI: bootstrap with 10,000 resamples of pairs (`agreement_bootstrap.py`).

I then wrote one-page guidelines (Appendix B; `docs/annotation_guidelines.md`), and both annotators used them to label the remaining 31 pairs, which had not been annotated before. Raw agreement rose to 97% ($\kappa = 0.65$; Table 5), but this is not evidence that the guidelines work. The annotators labelled 30 and 29 of the 31 pairs *no*, so κ rests on one or two pairs and its 95% interval spans 0.00 to 1.00. The two sheets' notes are closely similar in wording. Both annotators report that they labelled separately and by hand, so I treat the overlap as unexplained rather than as evidence of copying; I could not verify it further. The repository history for these two sheets begins with their upload on 7 November and does not show when or how they were filled in; each annotator keeps their original file. More importantly, the labels contradict the guidelines. Step 2 says that a description about the right product which adds an unsupported fact is *partial*, and in at least 20 of the 31 pairs the annotators' own notes describe exactly that (for example, a diamond ring is supported but the number of diamonds and the carat weight are not), yet the pairs were labelled *no*. Applied as written, most of these pairs would be *partial*. The round therefore shows that the guidelines, as worded, did not fix the boundary between *partial* and *no*; a next version needs worked examples at that boundary and

Data	Prefix init.	BLEU	NIST	MET	R-L	Cov.
E2E	scratch	57.1	5.17	64.9	59.4	78.1
ViGGO	scratch	32.7	3.79	48.4	44.9	76.2
ViGGO	E2E	**38.3**	**4.54**	**53.5**	47.4	**91.2**
ViGGO	E2E + act	37.0	4.42	53.3	**49.2**	82.3
Flipkart	scratch	0.7*	0.84	9.1	12.7	11.4
Flipkart	E2E	3.8	1.32	19.2	16.5	40.9
Flipkart	E2E + NR3	4.1	1.46	19.6	16.6	40.9

Table 6: Results with the same scorer as Table 1. ViGGO: 2,000 training pairs, 2 epochs, 100 test MRs; coverage over *name, release_year, developer*. Flipkart: 98 training pairs, 20 epochs, 12 test MRs. NR3: no repeated 3-gram within the generated text. *Smoothed; unsmoothed BLEU is 0. Single runs.

a short calibration round before labelling. My first-round labels were deliberately not revised. The 31-pair labels were not used to filter the data: these pairs stayed in the experiments (26 train, 3 dev, 2 test) even though 30 of my 31 labels for them are *no*, which the filter would have removed, so the filtered data rests on the first 100 labels only.

5 Results on New Data

System and evaluation. All new-data experiments use the `peft` reimplementation (Section 3.2), not the authors' code: the authors' code ran only in the patched Python 3.8 Colab environment, while the new-data runs, including the pilots and the warm start from the E2E prefix, were built on my `peft` script and run on the laptop CPU. This section therefore tests prefix-tuning as implemented in `peft`, which is 10.3 BLEU below the authors' code on E2E (57.1 vs. 67.4). The comparisons my conclusions rest on are within this one system (warm start vs. scratch, with and without the dialogue act, with and without the 3-gram constraint), so they do not depend on its absolute quality; the absolute ViGGO and Flipkart scores are probably lower than the authors' code would give. The evaluation also differs from the E2E replication: test sets are subsets (100 ViGGO MRs, 12 Flipkart MRs); each Flipkart MR has a single reference, whereas each E2E test MR has 7.4 on average, which lowers BLEU for the same output quality; and slot coverage is computed over different slots per dataset (Table 6). Decoding (greedy) and the scorer are the same as for the `peft` E2E run.

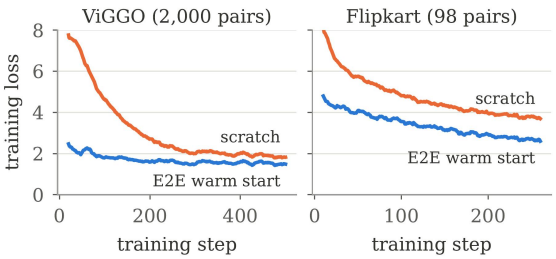


Figure 2: Training loss (moving average) for prefixes trained from scratch and warm-started from the E2E prefix.

5.1 ViGGO

ViGGO is harder than E2E for the same method (32.7 vs. 57.1 BLEU from scratch; Table 6), because many utterances are questions or opinions and references often add facts not in the MR (Figure 1). Starting from the E2E prefix instead of a random one starts training at a much lower loss (Figure 2) and helps on every metric: +5.6 BLEU and +15 points of slot coverage, with release year covered in 97% vs. 60% and developer in 77% vs. 40%. The warm-started model also invents fewer ESRB ratings: among the 78 test MRs without an *esrb* slot, 6 outputs mention a rating, against 21 for the cold start and 0 for the references. This is a small-scale echo of the paper's claim that prefixes transfer and extrapolate well. Giving the model the dialogue act raised the share of questions where the reference is a question (54% to 67%) and ROUGE-L, but lowered slot coverage (91% to 82%): the prefix trades content for form.

5.2 Flipkart

On Flipkart the method mostly fails: 3.8 BLEU with a warm start and essentially 0 from scratch. With only 98 training pairs the E2E warm start matters a great deal (coverage 41% vs. 11%), which is in line with the paper's low-data finding, but the outputs are poor. Four of the 12 cold-start outputs and 2 of the 12 warm-start outputs fall into repetition loops ("Brass, Brass, Brass, ..."). I added a decoding constraint that forbids repeating any 3-gram *within the generated text*; it removed all loops without hurting coverage. The built-in Hugging Face option (`no_repeat_ngram_size`) could not be used because it also counts the prompt, which blocked copying the product name (name coverage 42% to 0%). With only 12 test MRs these scores are very noisy: differences of a few tenths of a BLEU point (3.8 vs. 4.1 with the 3-gram con-

Data (n)	Fluency	Omit.	Halluc.	Repet.	Faithful
E2E (40)	4.55	1.62	12%	12%	8%
ViGGO (20)	3.90	1.05	40%	15%	25%
Flipkart (12)	2.58	2.08	58%	75%	0%

Table 7: LLM-judge results. Omit.: mean MR values not expressed per output. Faithful: no omission and no hallucination. For Flipkart, omissions count only describable attributes (type, colour, material, pattern, occasion, brand).

straint) are within noise, and only the large gaps (coverage 41% vs. 11%, the loops) should be read as real. The main problem is the data, not the method: when two-thirds of the training targets add unsupported facts, the prefix learns to invent plausible product details, which the LLM judge confirms below.

5.3 LLM-as-judge

Word overlap cannot tell an omission from an invention, so I added an LLM judge as a second, faithfulness-oriented evaluation layer, with the slot coverage check as a cheap automatic proxy and human annotation as the reference point. The judge (Claude Opus 5.5, used through the Claude app on 2 November 2026) received each MR and output with a fixed rubric (Appendix A) and returned: fluency (1–5), the number of MR values not expressed, whether any fact is invented or contradicted, whether the output repeats itself, and, for ViGGO, whether it fits the dialogue act. I sampled 40 E2E and 20 ViGGO outputs (random seed 1) and used all 12 Flipkart outputs of the warm-start run. All ratings are in `results/llm_judge_ratings.csv`.

Table 7 ranks the three datasets in the same order as BLEU, but adds what the metrics hide. On E2E the outputs are fluent (4.55) but only 3 of 40 express every attribute: the judge counts all eight attributes, including *customer rating* and *familyFriendly*, which slot coverage ignores, so 75.5% of all values are expressed against 81% slot coverage on the same sample. Five E2E outputs hallucinate, e.g. an invented "5 out of 5" rating or "low priced" for *customer rating[low]*. On ViGGO, slot coverage is 92% on the sample, yet 8 of 20 outputs invent or contradict a fact ("developed by E 10+", "not available on PlayStation, Xbox or PC" when the MR says it is), and the hallucinated outputs still have 88% coverage: coverage simply cannot see added content. Some ViGGO inventions copy a habit of the references, which also mention platforms absent from the MR. Only 14 of 20 ViGGO outputs fit their dialogue act. On Flipkart no output is faithful; 7 of 12 invent details (a top described as "casual jeans", "0.6 carat" diamonds), which matches the human finding that most targets are only *partially* supported.

The judge has clear limits, and I use it only as a diagnostic. It is a single model in a single pass, it has not been validated against human ratings of these outputs, and it was not blind to the aggregate metrics. It is also not independent of the project: the same assistant (Claude) ran the Colab replication and drafted this report, so it may share the report's expectations. For these reasons no conclusion in this report rests on the judge alone: the Flipkart conclusion rests on the human annotation (Section 4.3), and the ViGGO transfer result on BLEU and slot coverage (Table 6). The judge's role is to separate omission from hallucination and to name error types that slot coverage cannot see (invented ratings, platforms, diamond weights). Validating it would need human hallucination labels on the same outputs and a fresh session given only the rubric, MR and output.

6 Ethical Considerations

The original work. Prefix-tuning lowers the storage and compute cost of adapting large models, which reduces energy use and makes adaptation accessible to groups without large hardware budgets; my whole project ran on a laptop CPU. The risks come from the frozen LM it relies on. GPT-2 was trained on web text and reproduces social biases (Sheng et al., 2019), and a frozen model keeps those biases because only the prefix is trained (Bender et al., 2021). Data-to-text generation is used where readers trust the text to reflect a database, so hallucination (Ji et al., 2023) is a harm in itself: my E2E and ViGGO outputs invent ratings and platforms. The paper evaluates with word-overlap metrics that are known to correlate weakly with human judgements (Novikova et al., 2017a), so reported gains need not mean more faithful text. Finally, its datasets were written by crowd workers, whose pay and conditions are not part of the paper.

Publishing on the new data. The *Australian Code for the Responsible Conduct of Research* (National Health and Medical Research Council et al., 2018) sets out principles that include honesty, rigour, transparency, fairness, respect, recognition

and accountability, and responsibilities for data management, attribution and peer review. For this project they imply the following. *Attribution and licensing.* The Flipkart text was scraped from a commercial website and redistributed under CC BY-SA 4.0. Any release of my derived dataset must credit PromptCloud and the Hugging Face uploader and use the same ShareAlike licence, and the descriptions remain the sellers' and brands' text, not mine. The sellers wrote them to sell products, not expecting reuse as research data, and a licence on the redistributed copy does not settle whether the original collection respected Flipkart's terms of use, which I did not check and which would need checking before publication. A datasheet (Gebru et al., 2021) should document the filtering, which removed 87% of products and therefore biases the data towards categories with prose descriptions (clothing, jewellery). *Privacy.* The data holds product listings with brand names but no customer or personal data; this should be checked before release. *Human participants.* The second annotator is a human participant. A publication would need informed consent and, depending on the institution, ethics review under the National Statement on Ethical Conduct in Human Research (National Health and Medical Research Council et al., 2023), even for this low-risk task. *Honest reporting.* Rigour and honesty mean reporting the low agreement and the failed Flipkart results rather than tuning until they look good, and not re-labelling pairs after seeing the disagreement. *Downstream harm.* A model trained on this data invents product facts such as diamond weights. Deployed in a shop, it could produce misleading product claims, so the data is suitable for studying unfaithfulness, not for training a production system. *AI use.* The unit allows AI use; the LLM judge and AI writing assistance are disclosed in this report, and I remain responsible for every claim and reference.

7 Conclusions and Reflections

The replication largely succeeded, within clear limits. Once five environment problems were fixed, the authors' own code trained a GPT-2 medium prefix in 31 minutes on a free Colab GPU and reached 67.4 BLEU on E2E after one epoch, against 69.7 in the paper after five. My lightweight `peft` reimplementation on GPT-2 small and a laptop CPU reached 57.1 BLEU; the gap to the authors' code has identifiable causes (model size, training budget, no MLP reparameterisation, greedy decoding) and is concentrated in dropped attributes, above all the area (23% vs. 97% coverage). Its limits are that only E2E was replicated, the authors'-code run used one epoch and the general hyperparameters, its outputs were not kept, and every result is a single run. Most of the effort of replicating a five-year-old codebase went into its environment and into settings hidden in folder names, not into the method.

The method generalises well to a structurally similar new domain: on ViGGO, in my `peft` reimplementation, an E2E-trained prefix transfers and beats a fresh one on every metric. It does not generalise to my product data, but the reason is the data. Annotation and the LLM judge both show that the targets are written to sell, not to describe the MR.

Three reflections. First, dataset construction mattered more than any model choice. A better design would pair each product with a description written or edited to match its MR, as E2E's crowd workers did, even at a much smaller size. Second, annotation was harder than expected: an undefined *partial* label gave $\kappa = 0.03$ over 100 pairs, and guidelines should be written and piloted before annotation, not after, as I did. Third, faithfulness needs its own evaluation. BLEU, ROUGE and even slot coverage reward copying and miss invention, while an LLM judge exposed hallucinations that no automatic metric I computed could see.

Acknowledgements and Use of AI

I thank the second annotator, who labelled all 131 Flipkart pairs: the first 100 in two batches of 50 with only the label names, and the remaining 31 with the written guidelines. I chose the paper and the project design, ran all `peft` experiments, built the Flipkart data and annotated all 131 pairs (100 with only the label names, 31 with the guidelines), and checked the numbers and the text. Generative AI (Claude, Anthropic) was used as permitted for this assessment: at my request it wrote the code for the `peft` experiments and data handling (`train_prefix.py`, `build_flipkart.py`, `filter_by_annotation.py` and the evaluation and agreement scripts), ran the Colab replication of the authors' code in my browser session (Section 3.1), produced the LLM-judge ratings (Section 5.3), wrote `score_authors_output.py` and the Colab

notebook, and drafted the text of this report from my results. I have read every cited work.

References

Satanjeev Banerjee and Alon Lavie. 2005. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, pages 65–72, Ann Arbor, Michigan.

Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, pages 610–623.

Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46.

George Doddington. 2002. Automatic evaluation of machine translation quality using n-gram co-occurrence statistics. In *Proceedings of the Second International Conference on Human Language Technology Research*, pages 138–145.

Ondřej Dušek, Jekaterina Novikova, and Verena Rieser. 2020. Evaluating the state-of-the-art of end-to-end natural language generation: The E2E NLG challenge. *Computer Speech & Language*, 59:123–156.

Claire Gardent, Anastasia Shimorina, Shashi Narayan, and Laura Perez-Beltrachini. 2017. The WebNLG challenge: Generating text from RDF data. In *Proceedings of the 10th International Conference on Natural Language Generation*, pages 124–133, Santiago de Compostela, Spain. Association for Computational Linguistics.

Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. 2021. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92.

Sebastian Gehrmann et al. 2021. The GEM benchmark: Natural language generation, its evaluation and metrics. In *Proceedings of the 1st Workshop on Natural Language Generation, Evaluation, and Metrics (GEM 2021)*, pages 96–120, Online. Association for Computational Linguistics.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2790–2799.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12).

Juraj Juraska, Kevin Bowden, and Marilyn Walker. 2019. ViGGO: A video game corpus for data-to-text generation in open-domain conversation. In *Proceedings of the 12th International Conference on Natural Language Generation*, pages 164–172, Tokyo, Japan. Association for Computational Linguistics.

J. Richard Landis and Gary G. Koch. 1977. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174.

Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, Online. Association for Computational Linguistics.

Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597, Online. Association for Computational Linguistics.

Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain.

Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. 2022. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft.

Linyong Nan, Dragomir Radev, Rui Zhang, Amrit Rau, Abhinand Sivaprasad, Chiachun Hsieh, Xiangru Tang, Aadit Vyas, Neha Verma, Pranav Krishna, Yangxiaokang Liu, Nadia Irwanto, Jessica Pan, Faiaz Rahman, Ahmad Zaidi, Mutethia Mutuma, Yasin Tarabar, Ankit Gupta, Tao Yu, and 5 others. 2021.

DART: Open-domain structured data record to text generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 432–447, Online. Association for Computational Linguistics.

Shashi Narayan, Shay B. Cohen, and Mirella Lapata. 2018. Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1797–1807, Brussels, Belgium. Association for Computational Linguistics.

National Health and Medical Research Council, Australian Research Council, and Universities Australia. 2018. Australian code for the responsible conduct of research, 2018. https://www.nhmrc.gov.au/about-us/publications/australian-code-responsible-conduct-research-2018.

National Health and Medical Research Council, Australian Research Council, and Universities Australia. 2023. National statement on ethical conduct in human research 2023. https://www.nhmrc.gov.au/about-us/publications/national-statement-ethical-conduct-human-research-2023.

Jekaterina Novikova, Ondřej Dušek, Amanda Cercas Curry, and Verena Rieser. 2017a. Why we need new evaluation metrics for NLG. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 2241–2252, Copenhagen, Denmark. Association for Computational Linguistics.

Jekaterina Novikova, Ondřej Dušek, and Verena Rieser. 2017b. The E2E dataset: New challenges for end-to-end generation. In *Proceedings of the 18th Annual SIGdial Meeting on Discourse and Dialogue*, pages 201–206, Saarbrücken, Germany. Association for Computational Linguistics.

Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. Technical report, OpenAI.

Emily Sheng, Kai-Wei Chang, Premkumar Natarajan, and Nanyun Peng. 2019. The woman worked as a babysitter: On biases in language generation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3407–3412, Hong Kong, China. Association for Computational Linguistics.

Ramakrishna Vedantam, C. Lawrence Zitnick, and Devi Parikh. 2015. CIDEr: Consensus-based image description evaluation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4566–4575.

A LLM-Judge Rubric

The judge was not run through an API with a stored prompt: Claude (Opus 5.5) rated the 72 outputs interactively in a chat session on 2 November 2026, given the MR, the output and the rubric below, which is therefore the complete instruction. No sampling settings apply, and there is no separate prompt file or screenshot; the ratings and a note for each are in `results/llm_judge_ratings.csv`. For each item the judge saw the MR and the generated text and answered: (1) **Fluency** 1–5: 5 natural and grammatical; 3 understandable but awkward or repetitive; 1 broken or looping. (2) **Omitted values**: how many MR values the text does not express (E2E/ViGGO: every slot; Flipkart: type, colour, material, pattern, occasion, brand). (3) **Hallucination** yes/no: the text states a fact that is absent from or contradicts the MR (opinion words such as "stylish" do not count). (4) **Repetition** yes/no: a phrase or fact is repeated. (5) **ViGGO only, act fit** yes/no: questions for *request, verify_attribute, suggest, confirm*; an opinion for *give_opinion*; a statement for *inform*. A short justification was recorded for every judgement that was not perfect.

B Annotation Guidelines (written after the first round; used for the 31-pair round)

Read the MR and the description. Step 1: if the description does not mention the product (its type, name or brand), label **no**. Step 2: if it states any concrete fact absent from the MR (colour, material, number, size, feature, origin), label **partial**. Step 3: otherwise label **yes**; opinion words alone do not make a description partial. Give a short reason in the notes, and label without seeing the other annotator's labels. Examples from the labelled pairs: "…this black top will make you look elegant" with no colour in the MR is *partial*; "The countdown is on! Time to select gifts …" is *no*.

C Reproducing the Results

The authors'-code run, with its training log, timings and scores, is in `colab/original_code_run_with_outputs.ipynb` (Colab, T4 GPU); `colab/original_code_attempt.ipynb` is the same code without outputs. The main `peft` commands are below; the README lists the commands for every other run (ViGGO, Flipkart, the beam-search ablation, the official-scorer check `official_e2e_scores.py`, `score_authors_output.py`), and each run's exact arguments are stored in `results/<run>/results.json`.

```
pip install -r requirements.txt
git clone https://github.com/tuetschek/e2e-dataset \
  data/e2e
python src/train_prefix.py \
  --train_csv data/e2e/trainset.csv \
  --test_csv data/e2e/testset_w_refs.csv \
  --n_train 4000 --epochs 2 --lr 0.03 \
  --prefix_len 10 --n_eval 200 \
  --out_dir results/e2e_short
python src/train_prefix.py \
```

```
  --train_csv data/e2e/trainset.csv \
  --test_csv data/e2e/testset_w_refs.csv \
  --init_adapter results/e2e_short/adapter \
  --epochs 0 --n_train 8 --n_eval 630 \
  --out_dir results/e2e_full
python src/agreement_bootstrap.py \
  data/newdata/annotation_sheet.csv \
  data/newdata/rater2_batch1.csv \
  data/newdata/rater2_batch2.csv
```